SkillNavora

STAGE 3 HANDBOOK

Stage 3: Agent development

Outcome: Build tool-using agents and automated workflows. Topics: LangChain, LangGraph, CrewAI, Tool calling, MCP (Model Context Protocol), Agent memory, Multi-agent workflows, Human-in-the-loop.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Stage 3: Agent development

Outcome: Build tool-using agents and automated workflows

This handbook covers each topic of Stage 3 with a definition, why it matters, how it works, step-by-step instructions, example code or templates, common mistakes, interview questions with answers, a hands-on exercise and official resources.

Topics in this stage

1. LangChain
2. LangGraph
3. CrewAI
4. Tool calling
5. MCP (Model Context Protocol)
6. Agent memory
7. Multi-agent workflows
8. Human-in-the-loop

STAGE 3 · TOPIC 1

LangChain

DEFINITION

> LangChain is an open-source framework (Python and JavaScript) that provides standard building blocks for LLM applications: chat model wrappers for many providers, prompt templates, output parsers, tools, retrievers, document loaders and agents.

WHY IT MATTERS

> It lets you swap providers with one line, reuse hundreds of integrations (vector stores, loaders, tools) and build RAG and agents faster. Many customer codebases already use it, so FDEs must read and debug it.

HOW IT WORKS

- Chat models: ChatAnthropic, ChatOpenAI, ChatGoogleGenerativeAI share one interface (.invoke, .stream, .batch).
- Prompt templates insert variables into messages.
- Runnables chain steps with the | operator (prompt | model | parser).
- Tools are Python functions decorated with @tool; models call them through bind_tools.
- Retrievers return relevant documents from vector stores for RAG.
- LangChain's agent features are built on LangGraph; LangSmith provides tracing and evaluation. Framework APIs change between versions; check the linked official docs for the version you install.

STEP-BY-STEP

1. pip install langchain langchain-anthropic (or langchain-openai).
2. Create a chat model and call .invoke with a question.
3. Build a prompt | model | parser chain for structured extraction.

4. Define a tool with @tool and bind it to the model.
5. Turn on tracing in LangSmith to see each step.

EXAMPLE CODE / TEMPLATE

```
# pip install langchain langchain-anthropic
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.tools import tool

llm = ChatAnthropic(model='claude-opus-5-5', max_tokens=1500)

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You summarise property listings for buyers in {n} bullet points.'),
    ('human', '{listing}'),
])
chain = prompt | llm | StrOutputParser()
print(chain.invoke({'n': 3, 'listing': '3BHK, 1650 sq ft, 12th floor, east facing, 2 parking, near
metro'}))

@tool
def emi(principal: float, annual_rate: float, years: int) → float:
    """Monthly EMI for a home loan."""
    r, n = annual_rate / 12 / 100, years * 12
    return round(principal * r * (1 + r) ** n / ((1 + r) ** n - 1), 2)

reply = llm.bind_tools([emi]).invoke('EMI on 60 lakh at 8.5% for 20 years?')
print(reply.tool_calls)  # the model asks to call emi(...); your code runs it
```

COMMON MISTAKES

- Using deep abstractions you cannot debug; trace every step.
- Copying tutorial code written for an older LangChain version.
- Adding LangChain when a direct SDK call would be simpler.

INTERVIEW QUESTIONS AND ANSWERS

Q. When would you use LangChain instead of the provider SDK directly?

A. When you need many integrations (loaders, vector stores, multiple providers) or want to switch providers easily; for a simple single-provider call, the SDK is simpler and easier to debug.

Q. What is a Runnable/chain?

A. A composable step with a standard interface (invoke, stream, batch); chaining them with | passes each output into the next step.

HANDS-ON EXERCISE

> Build a LangChain RAG chain over 10 PDFs with a retriever and citations, and inspect a trace in LangSmith.

OFFICIAL AND RECOMMENDED RESOURCES

- LangChain docs (official) https://docs.langchain.com/oss/python/langchain/overview
- LangChain Anthropic integration https://docs.langchain.com/oss/python/integrations/chat/anthropic
- LangSmith docs https://docs.langchain.com/langsmith/home

LangGraph

DEFINITION

LangGraph is a library for building stateful, controllable agents and workflows as graphs: nodes are steps (functions or model calls), edges decide what runs next, and a shared state object carries data between them. It supports persistence (checkpoints), streaming and human-in-the-loop pauses.

WHY IT MATTERS

Enterprise agents need predictable control flow, retries, approvals and the ability to resume after failures. LangGraph makes the flow explicit, which is easier to test, debug and explain to customers than a free-running agent.

HOW IT WORKS

- State: a typed dictionary that every node reads and updates.
- Nodes: Python functions that take state and return updates.
- Edges: fixed (A → B) or conditional (a router function chooses the next node).
- START and END mark entry and exit.
- Checkpointers save state after each step so a run can pause (for approval) and resume.
- Cycles allow agent loops (model → tool → model) with limits. Framework APIs change between versions; check the linked official docs for the version you install.

STEP-BY-STEP

1. pip install langgraph langchain-anthropic.
2. Define the state (TypedDict) for a lead-triage workflow.
3. Write nodes: classify, enrich, route_hot, route_cold.
4. Add a conditional edge after classify.
5. Compile and invoke with a sample lead; print the final state.
6. Add a checkpointer and an interrupt before sending any message to the customer.

EXAMPLE CODE / TEMPLATE

```
# pip install langgraph langchain-anthropic
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model='claude-opus-5-5', max_tokens=500)

class Lead(TypedDict, total=False):
    message: str
    intent: str
    action: str

def classify(state: Lead) -> Lead:
    out = llm.invoke(f"Reply with one word, buy, rent or other: {state['message']}").content
    return {'intent': out.strip().lower()}

def hot(state: Lead) -> Lead:
    return {'action': 'Assign to senior agent and call within 5 minutes'}
```

```
def nurture(state: Lead) → Lead:
    return {'action': 'Add to weekly email nurture'}

def route(state: Lead) → str:
    return 'hot' if state['intent'] in ('buy', 'rent') else 'nurture'

g = StateGraph(Lead)
g.add_node('classify', classify)
g.add_node('hot', hot)
g.add_node('nurture', nurture)
g.add_edge(START, 'classify')
g.add_conditional_edges('classify', route, {'hot': 'hot', 'nurture': 'nurture'})
g.add_edge('hot', END)
g.add_edge('nurture', END)
app = g.compile()
print(app.invoke({'message': 'Want to buy a 2BHK in Pune this month'}))
```

COMMON MISTAKES

- Putting everything in one giant node.
- Unbounded cycles with no step limit.
- Storing large documents in state instead of references.
- No checkpointer, so long runs cannot resume or pause for approval.

INTERVIEW QUESTIONS AND ANSWERS

Q. Why model an agent as a graph?

A. Explicit steps and routing make behaviour predictable, testable and auditable; you can add approvals, retries and persistence at specific points instead of trusting a free-form loop.

Q. How does human-in-the-loop work in LangGraph?

A. With a checkpointer, the graph can interrupt before or inside a node, save state, wait for human input, then resume from the saved state with the human's decision.

HANDS-ON EXERCISE

> Build a LangGraph claims workflow: extract fields, validate, auto-approve small claims, and interrupt for human approval above a threshold.

OFFICIAL AND RECOMMENDED RESOURCES

- LangGraph docs (official) https://docs.langchain.com/oss/python/langgraph/overview
- LangGraph: interrupts (human-in-the-loop) https://docs.langchain.com/oss/python/langgraph/interrupts
- LangChain Academy (free courses) https://academy.langchain.com/

CrewAI

DEFINITION

> CrewAI is an open-source Python framework for multi-agent systems where each agent has a role, goal and backstory, works on tasks with expected outputs, and agents collaborate as a 'crew' run sequentially or hierarchically. Flows add event-driven, stateful control.

WHY IT MATTERS

> It is a fast way to prototype role-based teams (researcher, analyst, writer) and is popular with automation agencies. FDEs should know when its simplicity helps and when a controlled graph is better.

HOW IT WORKS

- Agent: role, goal, backstory, tools and the LLM it uses.
- Task: description, expected_output and the agent responsible.
- Crew: the list of agents and tasks plus a process (sequential or hierarchical with a manager).
- Tools: built-in (search, scraping, file reading) or custom Python functions.
- kickoff() runs the crew and returns the final output; outputs of earlier tasks feed later ones.
- Flows chain crews and plain code with state and conditions. Framework APIs change between versions; check the linked official docs for the version you install.

STEP-BY-STEP

1. pip install crewai.
2. Set ANTHROPIC_API_KEY (or another provider key).
3. Define a market researcher and a report writer agent.
4. Define two tasks with clear expected outputs.
5. Create the crew, run kickoff with inputs, and review the result.
6. Add a custom tool and limit iterations.

EXAMPLE CODE / TEMPLATE

```
# pip install crewai
from crewai import Agent, Task, Crew, Process

LLM = 'anthropic/claude-opus-5-5'  # provider/model string

researcher = Agent(
    role='Real-estate market researcher',
    goal='Summarise demand drivers for {area}',
    backstory='You analyse housing markets in Indian cities and cite assumptions.',
    llm=LLM, max_iter=5,
)
writer = Agent(
    role='Client report writer',
    goal='Write a one-page buyer brief',
    backstory='You write clear, jargon-free briefs for first-time buyers.',
    llm=LLM,
)
research = Task(description='List 5 demand drivers and 3 risks for buying in {area}.',
                expected_output='Two bullet lists with one-line explanations', agent=researcher)
```

```
brief = Task(description='Turn the research into a buyer brief with a recommendation.',
             expected_output='A 300-word brief with headings', agent=writer)

crew = Crew(agents=[researcher, writer], tasks=[research, brief], process=Process.sequential)
print(crew.kickoff(inputs={'area': 'Whitefield, Bengaluru'}))
```

COMMON MISTAKES

- Too many agents for a simple task, multiplying cost and errors.
- Vague expected outputs, so agents drift.
- No iteration limits or cost monitoring.
- Presenting generated research as fact without sources.

INTERVIEW QUESTIONS AND ANSWERS

Q. CrewAI vs LangGraph?

A. CrewAI is quicker for role-based collaboration with less code; LangGraph gives explicit, fine-grained control over state, routing, persistence and approvals, which suits complex production workflows.

Q. How do you keep a multi-agent crew reliable?

A. Clear roles and expected outputs, few agents, tool restrictions, iteration limits, evaluation of final outputs and logging every step.

HANDS-ON EXERCISE

> Build a two-agent crew that turns a customer's meeting notes into a requirements document and a risk list, then evaluate the outputs against a checklist.

OFFICIAL AND RECOMMENDED RESOURCES

- CrewAI docs (official) https://docs.crewai.com/
- CrewAI GitHub https://github.com/crewAIInc/crewAI

Tool calling

DEFINITION

Tool calling (function calling) lets a model request that your code run a named function with structured arguments. The model decides when a tool is needed; your application executes it and returns the result; the model then continues.

WHY IT MATTERS

Tools turn a model from a text generator into a system that can look up orders, query databases, create tickets and send messages. Almost every enterprise agent is built on tool calling, so safe, well-designed tools are central to FDE work.

HOW IT WORKS

- Define each tool: a clear name, a description of when to use it, and a JSON schema for its inputs.
- Send the tools with the request. If a tool is needed, the model returns a tool call (stop_reason tool_use for Claude) instead of a final answer.
- Your code validates the arguments, checks permissions, runs the function and returns the result (or an error) referencing the call ID.
- The model reads the result and either calls another tool or answers.
- Strict schemas (strict: true on Claude tools) guarantee arguments match the schema.
- Side-effecting tools (payments, emails, deletes) need confirmation and audit logs.

STEP-BY-STEP

1. Write the business function (get_order_status).
2. Describe it as a tool with a schema.
3. Implement the loop: call model → run tools → return results → repeat until end_turn.
4. Add argument validation and a maximum number of loop iterations.
5. Log every tool call with inputs, outputs and timing.

EXAMPLE CODE / TEMPLATE

```
import anthropic, json
client = anthropic.Anthropic()

ORDERS = {'A100': 'Shipped, arrives Friday', 'A101': 'Payment pending'}

def get_order_status(order_id: str) -> str:
    return ORDERS.get(order_id, 'Order not found')

tools = [{
    'name': 'get_order_status',
    'description': 'Look up the delivery status of a customer order by its ID, e.g. A100.',
    'input_schema': {'type': 'object',
                     'properties': {'order_id': {'type': 'string', 'pattern': '^A[0-9]{3}$'}},
                     'required': ['order_id'], 'additionalProperties': False},
    'strict': True,
}]
messages = [{'role': 'user', 'content': 'Where is my order A100?'}]

for _ in range(5):  # safety limit on loop iterations
```

```
    resp = client.messages.create(model='claude-opus-5-5', max_tokens=2000, tools=tools,
messages=messages)
    if resp.stop_reason != 'tool_use':
        print(''.join(b.text for b in resp.content if b.type == 'text'))
        break
    messages.append({'role': 'assistant', 'content': resp.content})
    results = []
    for block in resp.content:
        if block.type == 'tool_use' and block.name == 'get_order_status':
            out = get_order_status(**block.input)
            results.append({'type': 'tool_result', 'tool_use_id': block.id, 'content': out})
    messages.append({'role': 'user', 'content': results})  # all results in one message
```

COMMON MISTAKES

- Vague tool descriptions, so the model calls the wrong tool.
- Executing tool arguments as raw SQL or shell commands.
- No loop limit, so the agent can run forever.
- Dropping failed tool results instead of returning them with is_error.

INTERVIEW QUESTIONS AND ANSWERS

Q. How do you make tool use safe?

A. Least-privilege tools, strict schemas and argument validation, user-scoped data access, confirmation for side effects, loop and cost limits, and audit logs of every call.

Q. What makes a good tool definition?

A. A specific name, a description saying what it does and when to use it, a tight schema with required fields and allowed values, and concise, structured results.

HANDS-ON EXERCISE

> Give an assistant three tools (search_properties, book_visit, get_lead) and make book_visit require explicit user confirmation before execution.

OFFICIAL AND RECOMMENDED RESOURCES

- Claude: tool use overview https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview
- OpenAI: function calling https://platform.openai.com/docs/guides/function-calling
- Gemini: function calling https://ai.google.dev/gemini-api/docs/function-calling

MCP (Model Context Protocol)

DEFINITION

MCP is an open protocol that standardises how AI applications connect to tools and data. An MCP server exposes tools, resources and prompts; any MCP-compatible host (Claude Desktop, Claude Code, IDEs, agent frameworks) can connect to it through an MCP client.

WHY IT MATTERS

Instead of building a separate integration for every AI app, a company builds one MCP server for each system (CRM, ticketing, database) and reuses it everywhere, with consistent permissions. FDEs increasingly deliver MCP servers to customers.

HOW IT WORKS

- Host: the AI application. Client: the connector the host creates for each server. Server: your program exposing capabilities.
- Primitives: tools (actions the model can call), resources (data the app can read), prompts (reusable templates).
- Messages use JSON-RPC 2.0.
- Transports: STDIO for local servers launched by the host; Streamable HTTP for remote servers.
- Remote servers use OAuth-based authorisation; hosts ask users before running sensitive tools.
- The specification is versioned; check which version your SDK and hosts support.

STEP-BY-STEP

1. pip install mcp (official Python SDK).
2. Create a server with FastMCP and add two tools and one resource.
3. Run it locally and test with the MCP Inspector (npx @modelcontextprotocol/inspector).
4. Register it in Claude Desktop or Claude Code configuration.
5. Ask the assistant a question that needs your tool and watch the call.
6. Add input validation, authentication for remote use, and logging.

EXAMPLE CODE / TEMPLATE

```
# pip install mcp
from mcp.server.fastmcp import FastMCP

mcp = FastMCP('property-crm')

LISTINGS = [
    {'id': 'P1', 'area': 'Whitefield', 'bhk': 2, 'price_lakh': 85},
    {'id': 'P2', 'area': 'Whitefield', 'bhk': 3, 'price_lakh': 140},
    {'id': 'P3', 'area': 'HSR Layout', 'bhk': 2, 'price_lakh': 120},
]

@mcp.tool()
def search_listings(area: str, max_price_lakh: float, bhk: int | None = None) → list[dict]:
    """Find listings in an area under a budget, optionally filtered by BHK."""
    return [l for l in LISTINGS if l['area'].lower() == area.lower()
            and l['price_lakh'] ⩽ max_price_lakh and (bhk is None or l['bhk'] == bhk)]

@mcp.resource('listing://{listing_id}')
```

```
def get_listing(listing_id: str) -> str:
    """Full details of one listing."""
    return str(next((l for l in LISTINGS if l['id'] == listing_id), 'Not found'))

if __name__ == '__main__':
    mcp.run()  # STDIO transport by default
```

COMMON MISTAKES

- Installing MCP servers from unknown sources.
- Exposing write tools without confirmation or scopes.
- Returning huge payloads that flood the model's context.
- Ignoring the protocol version supported by the host.

INTERVIEW QUESTIONS AND ANSWERS

Q. What problem does MCP solve?

A. It replaces many custom, app-specific integrations with one standard protocol, so a tool or data source built once works across many AI applications.

Q. Tools vs resources vs prompts?

A. Tools are actions the model decides to call; resources are data the application reads and supplies as context; prompts are reusable templates users or apps select.

Q. How do you secure a remote MCP server?

A. HTTPS, OAuth-based authorisation with scoped tokens, per-user permissions on data, input validation, rate limits, audit logs and user consent for sensitive tools.

HANDS-ON EXERCISE

> Build an MCP server for a ticketing system (create_ticket, get_ticket, list_open) and connect it to Claude Desktop or Claude Code.

OFFICIAL AND RECOMMENDED RESOURCES

- Model Context Protocol (official) https://modelcontextprotocol.io/
- MCP architecture specification https://modelcontextprotocol.io/specification/2025-11-25/architecture
- MCP Python SDK https://github.com/modelcontextprotocol/python-sdk

Agent memory

DEFINITION

Agent memory is how an AI system keeps useful information beyond a single model call. Short-term memory is the state of the current task or conversation; long-term memory is facts, preferences and past outcomes stored outside the model and brought back when relevant.

WHY IT MATTERS

Models are stateless: each call only knows what you send. Without memory, assistants forget the customer's preferences, repeat questions and lose progress on long tasks; with too much memory, they get slow, costly and confused.

HOW IT WORKS

- Short-term: the messages and state of the current run (conversation history, task state, tool results).
- Long-term semantic memory: facts about users or entities ('prefers 3BHK near metro').
- Episodic memory: records of past interactions and outcomes.
- Procedural memory: learned instructions or playbooks.
- Storage: databases, key-value stores, vector stores (for retrieval by meaning), or files (Claude's memory tool reads and writes files through your code).
- Write policy (what to save), read policy (what to retrieve), and privacy rules (consent, retention, deletion) are design decisions.

STEP-BY-STEP

1. Decide what is worth remembering (stable preferences, decisions) and what is not (small talk, sensitive data).
2. Create a memory table keyed by user with fact, source and timestamp.
3. After each conversation, extract candidate facts with the model and save approved ones.
4. Before each conversation, load the user's top relevant facts into the context.
5. Provide a way for users to view and delete their memory.

EXAMPLE CODE / TEMPLATE

```
import sqlite3, time
db = sqlite3.connect('memory.db')
db.execute('CREATE TABLE IF NOT EXISTS memory (user_id TEXT, fact TEXT, created REAL)')

def remember(user_id: str, fact: str) → None:
    db.execute('INSERT INTO memory VALUES (?, ?, ?)', (user_id, fact, time.time()))
    db.commit()

def recall(user_id: str, limit: int = 10) → list[str]:
    rows = db.execute('SELECT fact FROM memory WHERE user_id = ? ORDER BY created DESC LIMIT ?',
(user_id, limit))
    return [r[0] for r in rows]

def forget(user_id: str) → None:            # honour deletion requests
    db.execute('DELETE FROM memory WHERE user_id = ?', (user_id,))
    db.commit()

remember('u42', 'Looking for a 3BHK near a metro station, budget about 1.4 crore')
```

```
profile = '\n'.join(recall('u42'))
system = f'Known facts about this customer (may be outdated, confirm if important):\n{profile}'
```

COMMON MISTAKES

- Saving everything, including sensitive data, without consent.
- Never expiring old facts, so the assistant acts on outdated preferences.
- Mixing users' memories through missing user IDs.
- Putting all memory in every prompt instead of retrieving relevant items.

INTERVIEW QUESTIONS AND ANSWERS

Q. Short-term vs long-term memory in agents?

A. Short-term memory is the working context of the current task; long-term memory persists across sessions in external storage and is retrieved selectively.

Q. What privacy issues does memory raise?

A. Storing personal data needs consent, purpose limits, retention rules, access control and user rights to see and delete it, in line with laws such as GDPR and India's DPDP Act.

HANDS-ON EXERCISE

> Add long-term memory to a property assistant: extract preferences after each chat, show them on a profile page, and let the user delete them.

OFFICIAL AND RECOMMENDED RESOURCES

- Claude: memory tool https://docs.claude.com/en/docs/agents-and-tools/tool-use/memory-tool
- LangGraph: memory concepts https://docs.langchain.com/oss/python/langgraph/memory

Multi-agent workflows

DEFINITION

A multi-agent workflow splits work among several model-driven agents or steps, each with a focused role, instructions and tools, coordinated by an orchestrator or a fixed sequence.

WHY IT MATTERS

Some tasks are too broad for one context window or benefit from specialisation and parallel work (researching many sources, processing many documents, drafting then reviewing). Done badly, multi-agent systems multiply cost and errors, so knowing when and how to use them matters.

HOW IT WORKS

- Prompt chaining: fixed sequence where each step's output feeds the next (extract → validate → draft).
- Routing: a classifier sends each input to a specialised handler.
- Parallelisation: several workers handle independent parts at once, then results are combined.
- Orchestrator–workers: a lead model plans sub-tasks dynamically and delegates them.
- Evaluator–optimiser: one agent produces, another critiques, and the first revises.
- Shared state, clear hand-off formats, limits and final verification keep the system controlled.

STEP-BY-STEP

1. Write the task as steps; check whether one agent with tools can do it (start simple).
2. If not, choose a pattern (chain, route, parallel, orchestrator, evaluator).
3. Define each agent's input, output format and tools.
4. Implement the orchestration in plain code first; add a framework only if needed.
5. Log each agent's input and output; evaluate the final result against a rubric.

EXAMPLE CODE / TEMPLATE

```
import anthropic, concurrent.futures as cf
client = anthropic.Anthropic()

def ask(system: str, prompt: str, model: str = 'claude-opus-5-5') -> str:
    r = client.messages.create(model=model, max_tokens=2000, system=system,
                               messages=[{'role': 'user', 'content': prompt}])
    return ''.join(b.text for b in r.content if b.type == 'text')

sections = ['Location and connectivity', 'Pricing trend', 'Builder track record', 'Legal checks']
property_notes = open('property_notes.txt', encoding='utf-8').read()

# Parallel workers (a cheaper model is fine for focused drafting)
with cf.ThreadPoolExecutor() as pool:
    drafts = list(pool.map(lambda s: ask(f'You write the "{s}" section of a buyer report.',
property_notes, 'claude-haiku-5-5'), sections))

# Reviewer combines and checks
report = ask('You are an editor. Merge the sections, remove contradictions, and flag any claim not
supported by the notes.',
             '\n\n'.join(f'## {s}\n{d}' for s, d in zip(sections, drafts)) + f'\n\n<notes>
{property_notes}</notes>')
print(report)
```

COMMON MISTAKES

- Using many agents where one would do.
- Agents passing long unstructured text to each other.
- No final verification step.
- No overall cost or time limit.

INTERVIEW QUESTIONS AND ANSWERS

Q. When is a multi-agent design justified?

A. When sub-tasks are independent and benefit from parallel work, need different tools or expertise, or exceed one context window; otherwise a single agent or simple chain is cheaper and more reliable.

Q. How do you debug a multi-agent system?

A. Trace each agent's inputs, outputs, tool calls and costs; test each agent alone with fixed inputs; then test the orchestration with recorded cases.

HANDS-ON EXERCISE

> Build an orchestrator that splits a 50-page tender document into sections, has workers summarise requirements in parallel, and has a reviewer produce a compliance checklist.

OFFICIAL AND RECOMMENDED RESOURCES

- Anthropic: building effective agents https://www.anthropic.com/engineering/building-effective-agents
- Anthropic: how we built our multi-agent research system https://www.anthropic.com/engineering/built-multi-agent-research-system
- OpenAI Agents SDK: handoffs https://openai.github.io/openai-agents-python/handoffs/

Human-in-the-loop

DEFINITION

> Human-in-the-loop (HITL) designs a system so that people review, approve, correct or take over at defined points: before risky actions, when confidence is low, or for a sample of outputs.

WHY IT MATTERS

> Enterprises accept AI agents only when consequential actions (payments, customer emails, data changes, credit or medical decisions) stay under human control with an audit trail. HITL also produces corrected examples that improve the system.

HOW IT WORKS

- Approval gates: the agent proposes an action and waits for a person to approve, edit or reject.
- Escalation: hand the case to a human when confidence is low, the request is out of scope or the customer asks.
- Review queues: people check outputs before release or sample them after.
- Thresholds: amount, risk category or model confidence decide which path a case takes.
- State must be saved while waiting (a checkpoint or database record) so the run can resume.
- Every decision is logged: who approved what, when, and with which information.

STEP-BY-STEP

1. List every action the agent can take and rate its risk.
2. Decide which actions need approval and the thresholds.
3. Store proposed actions in a pending table with all context.
4. Build a simple approval screen or Slack/Teams message with approve/reject.
5. Execute only approved actions; log the decision and outcome.
6. Review rejected proposals weekly to improve prompts and rules.

EXAMPLE CODE / TEMPLATE

```
import uuid, datetime
PENDING: dict[str, dict] = {}   # use a database table in production
AUDIT: list[dict] = []

def propose_refund(order_id: str, amount: float, reason: str) -> str:
    if amount <= 500:                       # low risk: auto-approve
        return execute_refund(order_id, amount, approver='auto')
    pid = str(uuid.uuid4())
    PENDING[pid] = {'order_id': order_id, 'amount': amount, 'reason': reason}
    return f'Refund of {amount} sent for manager approval (request {pid})'

def decide(pid: str, approver: str, approve: bool) -> str:
    req = PENDING.pop(pid)
    AUDIT.append({**req, 'approver': approver, 'approved': approve, 'at':
datetime.datetime.utcnow().isoformat()})
    return execute_refund(req['order_id'], req['amount'], approver) if approve else 'Refund rejected'

def execute_refund(order_id: str, amount: float, approver: str) -> str:
    # call the payment API here
    return f'Refunded {amount} on {order_id} (approved by {approver})'
```

COMMON MISTAKES

- Approval screens that lack the information needed to decide.
- Too many approvals, so people click 'approve' without reading.
- No timeout or reminder for pending approvals.
- Not learning from rejected proposals.

INTERVIEW QUESTIONS AND ANSWERS

Q. Where would you put human checkpoints in an AI claims process?

A. Before any payment above a threshold, for any denial, for low-confidence extractions, for cases flagged as unusual or potentially fraudulent, and for a random sample of auto-approved claims for quality control.

Q. How do you avoid approval fatigue?

A. Approve only risky actions, auto-handle low-risk ones, show concise evidence and the model's reasoning summary, batch similar items, and measure approval rates to tune thresholds.

HANDS-ON EXERCISE

> Add an approval gate to an email-sending agent: drafts go to a review queue, and only approved drafts are sent, with a full audit log.

OFFICIAL AND RECOMMENDED RESOURCES

- LangGraph: human-in-the-loop https://docs.langchain.com/oss/python/langgraph/interrupts
- Google PAIR: People + AI Guidebook https://pair.withgoogle.com/guidebook/
- NIST AI Risk Management Framework https://www.nist.gov/itl/ai-risk-management-framework